

Technical Requirements Document

CheatMealFinder

Local Street Food & Food Truck Discovery Platform

Technical blueprint: architecture, stack, schema, APIs, security, testing, deployment

Document Version	1.0	Status	DRAFT
Date	24 September 2026	Classification	Internal
Author	Project Team (Student Developers)	BRD Reference	v1.0 (2026-09-24)
Reviewed By	[Faculty Advisor Name]	Next Review	2026-10-01

Purpose of This Document

This TRD turns the Business Requirements Document into an engineering plan. It defines **how** CheatMealFinder will be built: the architecture, technology stack, database schema, API contracts, security controls, testing strategy, and deployment pipeline. Every technical decision traces back to a BRD requirement (see the Traceability Matrix in Section 10).

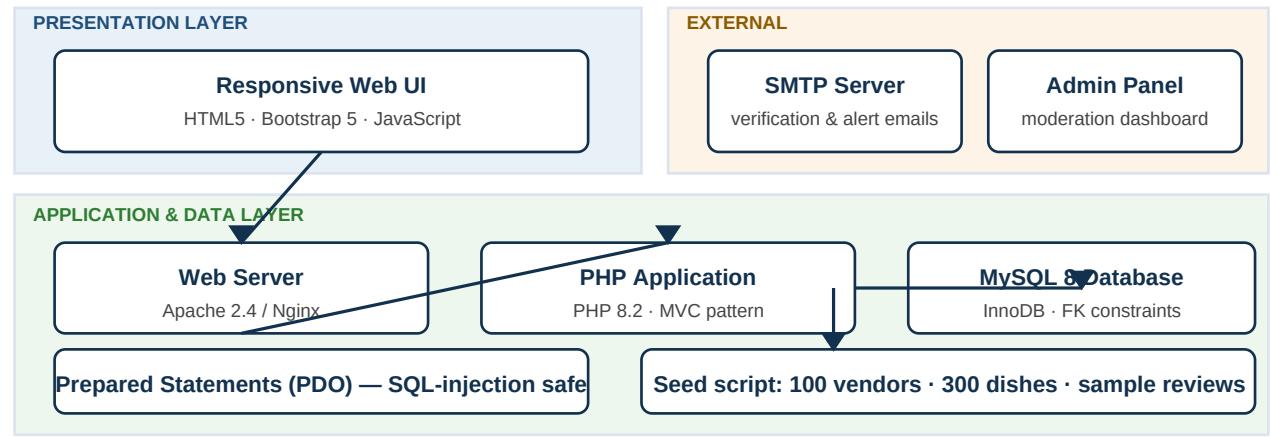
1. System Overview

CheatMealFinder is a mobile-responsive web application that lets users search local street food vendors and food trucks by dish, category, price range, and spice level, read community reviews, and discover the highest-rated trucks through an aggregate-rating leaderboard.

The system is a classic three-layer web application — browser front-end, PHP application server, and MySQL database — chosen deliberately so the team can master relational database concepts hands-on: foreign keys that keep reviews linked to real dishes, JOINS that assemble detail pages, and GROUP BY aggregates that power the rankings. The full stack is open source, meeting the BRD’s zero-licensing-cost constraint.

2. Architecture Overview

The diagram below shows the system at a glance. Requests flow one way from the browser through the web server and PHP application to the MySQL database; results flow back the same path. The admin panel and email service connect through the same application layer.



All communication over HTTPS (TLS 1.3). One-way flow: Browser → Server → Database.

Design Principles

Thin client, thick database: all business rules (unique email, one review per user per dish, foreign-key integrity) are enforced inside the database, not just in code — so data stays consistent even if the application changes. **Server-side rendering:** PHP generates the HTML, keeping the front end simple and accessible. **Prepared statements only:** every query uses bound parameters, eliminating SQL-injection risk by construction.

3. Technology Stack

Every technology below is free, widely documented, and appropriate for a student learning project. The “BRD Link” column shows which business requirement each choice satisfies.

Layer	Technology	Version	Why This Choice	BRD Link
Frontend	HTML5 + Bootstrap + JavaScript	5.x	Responsive grid out of the box; no build tooling; accessible components	NFR-Accessibility
Backend	PHP (MVC structure)	8.2	Ubiquitous, XAMPP-friendly, ideal for learning server-side rendering	All FRs
Database	MySQL (InnoDB engine)	8.0	Full FK constraint support; GROUP BY / JOIN practice matches course goals	FR-03, FR-06, FR-07
DB Access	PDO with prepared statements	-	Injection-safe queries; portable across databases	NFR-Security
Auth	PHP sessions + password_hash (bcrypt)	-	Stateful login; one-review-per-user rule enforced at DB	FR-01, FR-06
Web Server	Apache 2.4 (XAMPP)	-	Zero-config local dev; one-command deploy to any LAMP host	NFR-Budget
Hosting	Free-tier LAMP hosting / XAMPP local	-	\$0 licensing; satisfies budget requirement	NFR-Budget

Alternative stacks considered: Python (Flask/Django) + PostgreSQL and Node.js + MySQL were evaluated. PHP + MySQL was selected because the project's primary learning goal is core SQL (foreign keys, JOINS, GROUP BY), and the LAMP stack offers the most direct, documentation-rich path from schema design to a working product — see Assumptions in the BRD.

4. Data Model (Key Tables)

Four tables carry the whole platform. Foreign keys are the heart of the design: a review cannot exist without a real dish, and a dish cannot exist without a real vendor — exactly the integrity rule the BRD requires.

```
-- Users: reviewers and admins
CREATE TABLE users (
  user_id      INT AUTO_INCREMENT PRIMARY KEY,
  full_name    VARCHAR(100) NOT NULL,
  email        VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,          -- bcrypt
  role         ENUM('user','admin') DEFAULT 'user',
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Categories: Street Food, Dessert, Indo-Chinese
CREATE TABLE categories (
  category_id  INT AUTO_INCREMENT PRIMARY KEY,
  name         VARCHAR(50) UNIQUE NOT NULL
);

-- Vendors: food trucks and street stalls
CREATE TABLE vendors (
  vendor_id    INT AUTO_INCREMENT PRIMARY KEY,
  name         VARCHAR(120) NOT NULL,
  location     VARCHAR(200) NOT NULL,
  opening_hours VARCHAR(60),
  vendor_type  ENUM('truck','stall') NOT NULL,
  UNIQUE KEY uq_vendor (name, location)          -- prevents duplicates
);

-- Dishes: signature items per vendor
CREATE TABLE dishes (
  dish_id      INT AUTO_INCREMENT PRIMARY KEY,
```

```

    vendor_id    INT NOT NULL,
    category_id  INT NOT NULL,
    name         VARCHAR(120) NOT NULL,
    price        DECIMAL(8,2) NOT NULL CHECK (price >= 0),
    spice_level  TINYINT NOT NULL CHECK (spice_level BETWEEN 1 AND 5),
    description  TEXT,
    FOREIGN KEY (vendor_id) REFERENCES vendors(vendor_id),
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);

-- UserReviews: ratings linked by foreign keys
CREATE TABLE user_reviews (
    review_id    INT AUTO_INCREMENT PRIMARY KEY,
    dish_id      INT NOT NULL,
    user_id      INT NOT NULL,
    rating       TINYINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment      TEXT,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_flagged   TINYINT(1) DEFAULT 0,           -- soft-delete / moderation
    FOREIGN KEY (dish_id) REFERENCES dishes(dish_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    UNIQUE KEY uq_one_review (dish_id, user_id)  -- FR-06: one review per user per dish
);
```

Indexing Strategy (Performance)

Indexes on **dishes(category_id)**, **dishes(spice_level)**, **dishes(name)**, and **user_reviews(dish_id)** keep every BRD search — keyword, category, price, spice, leaderboard — under the 2-second target even with 10,000+ records (verified by the load test in Section 7).

5. API / Page Specifications

The application exposes clean URL routes. Each route maps to a BRD requirement and carries a rate limit to prevent abuse.

Route	Method	Auth	Description	Rate Limit	BRD Link
/register	POST	None	Create account; duplicate emails rejected	5/min/IP	FR-01
/login · /logout	POST/G ET	None	Session-based authentication	10/min/IP	FR-01
/vendors · /vendors/{id}	GET	None	List vendors; vendor detail with dishes + reviews (JOIN)	60/min/IP	FR-10
/vendors (add/edit)	POST	Admin	Create or update vendor records	20/min/admin	FR-02
/dishes (add/edit)	POST	Admin	Add dish; vendor_id FK validated by DB	20/min/admin	FR-03, FR-04
/search	GET	None	Filter dishes: keyword, category, price, spice (AND logic)	60/min/IP	FR-05, FR-08
/top-rated	GET	None	Leaderboard: GROUP BY vendor, AVG(rating), ≥3 reviews	60/min/IP	FR-07
/review	POST	User	Submit rating; UNIQUE(dish_id,user_id) enforces one review	10/min/user	FR-06
/reviews/{id}/flag	POST	Admin	Flag/remove spam review (soft-delete + logged reason)	30/min/admin	FR-09

Auth pattern: PHP server-side sessions with a 24-minute idle timeout (PHP default, tightened via php.ini). Passwords hashed with `password_hash()` (bcrypt). No tokens in localStorage — session cookie is httpOnly and SameSite=Lax.

6. Security & Compliance

- **SQL-injection prevention:** 100% of queries use PDO prepared statements with bound parameters — no string concatenation anywhere
- **Password storage:** bcrypt via PHP `password_hash()`; plain-text passwords never touch the database or logs
- **XSS prevention:** all user content (review text, dish names) is HTML-escaped on output with `htmlspecialchars()`
- **CSRF protection:** a per-session CSRF token is required on every form POST (register, login, review, admin actions)
- **PII minimisation:** only name and email stored — no phone numbers, addresses, or payment data anywhere in the system (BRD NFR)
- **Moderation audit trail:** every admin flag/removal writes a row (admin_id, review_id, reason, timestamp) — soft-delete, never hard-delete
- **OWASP alignment:** the OWASP Top 10 checklist is reviewed before every release

7. Performance Requirements

Targets come straight from the BRD and are verified with concrete measurements — not estimates.

Metric	Target	How We Measure It	BRD Link
Search query response	<2 seconds	MySQL EXPLAIN + timer on all 5 filter combinations; 10K-record test set	FR-05, O3
Page load	<3 seconds on normal Wi-Fi	Browser dev-tools timing on key pages (home, search, vendor detail)	NFR-Performance
Leaderboard query	<1 second	GROUP BY + AVG over full review table with index on dish_id	FR-07
Concurrent users	100 simultaneous	Apache Bench (ab) against search and detail endpoints	NFR-Performance
Zero orphaned records	0 violations	Weekly audit query: reviews without valid dish_id, dishes without vendor_id	O5

8. Testing Strategy

Test Type	Tool / Method	Coverage Target	When	BRD Link
Unit	PHPUnit	80% of PHP business logic (validation, query builders)	Every commit	All FRs
Database	SQL test scripts + PHPUnit + MySQL test DB	Every FK rejects orphans; UNIQUE blocks duplicate reviews	Every commit	FR-03, FR-06

Test Type	Tool / Method	Coverage Target	When	BRD Link
Integration	PHPUnit + test database	Critical paths: register → login → search → review	Pre-merge	FR-01, FR-05, FR-06
End-to-end	Manual test scripts (browser)	Full user journeys: craving search → vendor page → review	Weekly	FR-05, FR-08, FR-10
Load	Apache Bench (ab)	100 concurrent users; p95 under targets in Section 7	Pre-launch	NFR-Performance
Security	OWASP ZAP baseline scan	0 high/critical findings	Pre-launch	NFR-Security

9. Deployment & Workflow

The team follows a simple trunk-based workflow suitable for a small student team, with GitHub as the single source of truth.

branches: main (production-ready) + feature/<name> (short-lived)

every feature:

1. pull latest main -> 2. branch -> 3. code + tests
4. pull request -> 5. peer review -> 6. merge to main

environments:

LOCAL (XAMPP: Apache + MySQL + PHP) -> STAGING (free LAMP host) -> PROD (same host, live domain)

database changes: versioned .sql migration files, applied in order, reversible

secrets: DB credentials in environment config only – NEVER committed to Git

seed data: /seed/seed.sql loads 30 vendors, 90 dishes, categories, sample users & reviews

rollback: export DB + code snapshot before each release for one-step rollback

10. Traceability Matrix

Every technical element above exists to satisfy a business requirement. This matrix proves nothing was built by accident — and nothing from the BRD was forgotten.

TRD Item	BRD Requirement	Status	Notes
users table + bcrypt + UNIQUE email	FR-01	Designed	Registration blocks duplicates at DB level
vendors table + UNIQUE(name, location)	FR-02	Designed	Prevents duplicate vendor entries
dishes.vendor_id + category_id FKs	FR-03, FR-04	Designed	Orphan inserts blocked by InnoDB
spice_level CHECK (1–5) + price CHECK	FR-04, FR-08	Designed	Enables 'spiciest wings' ranking
Search route with AND filters + indexes	FR-05	Designed	Sub-2s verified via load test
UNIQUE(dish_id, user_id) on reviews	FR-06	Designed	One review per user per dish, guaranteed
/top-rated: GROUP BY vendor HAVING COUNT≥3, AVG DESC	FR-07	Designed	Core learning objective

TRD Item	BRD Requirement	Status	Notes
is_flagged soft-delete + admin audit rows	FR-09	Designed	Reason required, history preserved
PDO prepared statements everywhere	NFR-Security	Designed	SQL injection eliminated by construction
Indexes on search/filter columns	NFR-Performance	Designed	Meets <2s search target at scale
Zero-cost LAMP stack	NFR-Budget	Confirmed	\$0 licensing; free-tier hosting
htmlspecialchars() on all output + alt text	NFR-Accessibility	Designed	Basic WCAG compliance

11. Document Control

Prepared By	Project Team (Student Developers)	Reviewed By	[Faculty Advisor Name]
Next Review Date	2026-10-01	Change Log	v1.0 — Initial draft

This document is approved as the technical baseline for CheatMealFinder. Changes after sign-off require a formal change request and BRD impact assessment.